

Pre-Rerun Audit and Configuration Changes

IT4216 — Observability for Autoscaling Correctness in Kubernetes-Based Systems

Author	S.A.C.S. Indrejith (2020/ICT/65)
Supervisor	Suvin
Document version	1.0
Date	13 July 2026
Purpose	Comprehensive record of every configuration and methodology change made between the counted campaign and the ongoing rerun, with reasoning and predicted impact on the research outcomes.

1. Executive Summary

The counted campaign concluded on 11 July 2026 with 206 recorded HPA decisions across 92 runs of four workload patterns. During analysis two significant issues surfaced. First, the run-tagging logic in **build_master_dataset.py** contained a bug that attributed each Step and Burst run's post-load scale-up to the previous run, causing 71 of 206 decisions to be mistagged; SES was empty for those patterns and SRD was evaluated against the wrong runs' latency traces. Second, the four workload scripts were calibrated for the earlier 2 vCPU / 30 % HPA target pilot cluster and had never been recalibrated after the migration to the current 4 vCPU / 75 % target cluster, leaving them too small to meaningfully stress the current infrastructure (only 1 of 182 counted decisions had a sustained SLO breach).

Rather than defend the compromised results, we conducted a full audit of every subsystem in the pipeline, tightened calibration against the current cluster, and initiated a fresh rerun with a corrected methodology. This document records every change made in that pass, the reasoning behind each, and the anticipated impact on the research outcomes.

Scope of changes

The audit covered all seven subsystems of the experimental framework. Sixteen files were modified across the following areas:

- **Workload scripts and orchestrator** — amplitudes recalibrated for the current cluster; run numbers embedded in output filenames; NTP-sync pre-flight check; durable JSONL snapshot per batch
- **Kubernetes manifests** — pod memory raised, startupProbe added to protect JVM warm-up, explicit HPA behavior block written out for reproducibility
- **HPA watcher** — memory limit raised to eliminate OOM risk under bursty scaling
- **Analysis pipeline** — direction-aware tagging fix, SLO-sustained parameter converted from sample-count to time-based, CPU-based classifier renamed to LEGACY and dropped from primary flow, campaign start date now parameterised
- **Sample-app** — explicit JVM heap ceiling to prevent implicit container-aware defaults from shifting
- **Monitoring stack** — Prometheus switched from ephemeral emptyDir to persistent volume, retention raised from 3 to 14 days, chart version pinned
- **Droplet setup** — k3s and chart versions pinned, droplet size documentation corrected, Docker install override supported

Rerun status: complete

The 92-run counted rerun commenced at 10:58 UTC on 13 July 2026 and completed at approximately 09:22 UTC on 14 July 2026, roughly 22.5 hours end-to-end. All four pattern batches (Step, Burst, Ramp, Noisy) completed with the durable JSONL snapshot written at the end of each batch. The analysis pipeline (build_master_dataset → compute_srd → compute_ses → classify_decisions_v2) produced 280 tagged HPA decisions, of which 270 fall in counted (non-warm-up) runs. See Section 7 for the observed results and their comparison to the counted campaign.

2. Background and Motivation

2.1 What the counted campaign produced

The counted campaign ran 92 iterations across the four workload patterns (Step, Burst, Ramp, Noisy) with 3 warm-up runs per pattern discarded from analysis. The corrected results at the 500 ms SLO threshold with a sustained-breach requirement of 3 consecutive windows classified 182 counted decisions as follows:

Pattern	Correct & Timely	Correct but Late	Unnecessary	Total
Step	20 (50 %)	0	20 (50 %)	40
Burst	21 (52 %)	1 (3 %)	18 (45 %)	40
Ramp	50 (49 %)	0	52 (51 %)	102
Noisy	0	0	0	0
Total	91 (50 %)	1 (0.5 %)	90 (49 %)	182

Two features of this distribution flagged deeper issues. Only **one** decision in 182 was preceded by a sustained SLO breach, which is inconsistent with the methodology's four-bucket taxonomy of scaling correctness — the Correct-but-Late bucket is essentially empty. And the Noisy pattern produced **zero** counted decisions across 20 counted runs, meaning the analysis captured no data at all for one of four patterns.

2.2 The data-tagging bug

The function `tag_decisions` in `build_master_dataset.py` associated each HPA decision with the first run whose window `[k6_start, k6_end + 15 min]` contained the decision timestamp. The 15-minute post-k6 tail was intended to capture scale-down decisions that fire during HPA's default 5-minute scale-down stabilisation window, but with the first-match rule it also captured decisions belonging to the *next* run whose k6 stream had already begun. Empirically:

- 88 of 91 consecutive run pairs had overlapping windows under the 'end + 15 min' rule.
- 71 of 206 decisions were tagged to the wrong run.
- For Step and Burst patterns in particular, the initial post-load scale-up was attributed to the previous run, so SES before/after windows were computed against the previous run's k6 stream — producing empty SES values for those decisions.
- SRD was evaluated against the wrong latency trace, so any positive SRD reflected a spurious rather than real reaction delay.

A post-hoc external correction resolved the 206 rows, and the corrected CSVs shipped alongside a preserved backup of the pre-correction data. However, the buggy logic remained in the source code — a rerun using the same script would reproduce the same mistagging. This forced a code-level fix.

2.3 The workload calibration mismatch

Comment blocks in **calib-probe.js** and the four workload scripts documented that VU counts were calibrated for a 2 vCPU node running HPA at a 30 % CPU target with maxReplicas = 6 (the pilot configuration). The counted campaign ran on a 4 vCPU / 8 GiB droplet with HPA at a 75 % CPU target and maxReplicas = 10. The workload scripts were never resized to reflect the migration:

- Step and Ramp peaks of 15 and 20 VUs respectively produced only ~60 % CPU on the initial 2 replicas — below the 75 % trigger threshold.
- Burst's 25-VU peak briefly exceeded threshold on the first pod but the amplitude was insufficient to produce reliable 'transient triggers HPA' observations.
- Noisy's peaks of 22-35 requests per second were far below the amplitude needed to move the metrics-server averaged reading past 75 %, explaining the empty Noisy row.

The under-triggering is co-produced by the industry-representative 75 % HPA target (which is legitimately more conservative than the pilot's 30 %) and by the stale workload calibration (which is a documentation/tooling gap). The former is a defensible research choice; the latter is a mistake that must be corrected before reporting results.

2.4 Decision to rerun

In the 12 July 2026 supervisor correspondence, agreement was reached to rerun the counted campaign at the 75 % HPA target with the corrected pipeline and recalibrated workload amplitudes. Rather than execute a targeted patch, the opportunity was taken to run a full audit of the seven subsystems in the pipeline — the philosophy being 'close every blind hole we didn't catch, not just the ones we caught by accident'.

The remainder of this document records each subsystem's changes.

3. Detailed Changes by Subsystem

Each subsystem is presented with a summary of what changed, why the change was made, and the anticipated impact on the rerun's outcomes. Change blocks are colour-coded: blue 'WHY' explains the rationale for supervisor discussion, and green 'IMPACT ON RERUN' explains the expected effect on the results.

3.1 Workload scripts and orchestrator (06-load-tests)

Four k6 workload scripts and a bash orchestrator (**run-campaign.sh**) were updated. The July 2026 calibration probe against the current cluster informed the new amplitudes.

3.1.1 Peak-VU amplitudes recalibrated for the current cluster

The peak virtual-user (VU) count of each workload pattern was raised against the 4 vCPU / 75 % target cluster based on the recalibration probe. The Noisy pattern additionally shifted its per-second request targets from a 22-35 range to an 80-150 range and its VU allocation was raised to sustain the higher arrival rates.

Pattern (old peak)	Recalibrated peak
Step: 15 VUs	25 VUs
Burst: 25 VUs	35 VUs
Ramp: 20 VUs (linear over 8 min)	30 VUs (linear over 8 min)
Noisy: spikes 22-35 req/s, preAllocatedVUs=30, maxVUs=50	spikes 80-150 req/s, preAllocatedVUs=60, maxVUs=100

WHY

The counted campaign showed the pilot-era amplitudes never sustained enough per-pod CPU to trigger HPA meaningfully on the current cluster. The recalibration probe held load at 5, 15, 25 and 40 VUs for 2 minutes each and observed steady-state HPA CPU utilisation of approximately 5 %, 60 %, 75 % and 85 % respectively. Peaks were chosen so that each pattern reliably produces the transient behaviour it was designed to characterise: Step and Ramp with sustained scaling, Burst with a brief but stressful transient, and Noisy with high-amplitude short spikes that HPA may or may not react to.

IMPACT ON RERUN

The rerun should yield HPA scale-up events on every counted Step and Burst run and multiple pre-emptive scale-ups on Ramp. The Noisy pattern should now record decisions rather than remain empty; whether it triggers on every spike is itself a flapping-resistance observation. Overall counted decisions should rise from ~182 to somewhere in the 250-400 range.

3.1.2 Removed the 2026-07-03 'workload deliberately not tuned' note in Burst

The header comment in **burst-load.js** documented a 3 July 2026 decision to keep the workload amplitude at 25 VUs and to characterise HPA at 'the workload's natural saturation point'. That comment was replaced with a note recording the recalibration and its rationale.

WHY

The recalibration plan implicitly overrides the 3 July decision. Leaving the old rationale in place would create a documentation contradiction that a thesis reviewer would spot in the next audit. The old rationale is now noted in the diagnostic history section of the file — preserved but clearly marked as superseded.

IMPACT ON RERUN

No behavioural change beyond the amplitude update above — this is documentation hygiene. Its main purpose is to prevent the supervisor discussion from being sidetracked by an apparent inconsistency between the file's header comment and the recalibration decision.

3.1.3 NTP-sync pre-flight check in run-campaign.sh

The **check_cluster_health** function now refuses to start a run if the droplet's clock is not NTP-synchronised. It queries **timedatectl show -p NTPSynchronized** and fails fast if the answer is not **yes**.

WHY

The SRD and SES metrics depend on the k6 client's request timestamps and the droplet's Kubernetes API timestamps being aligned to within tens of milliseconds. Ubuntu 24.04's default systemd-timesyncd normally handles this, but a silent drift — for example after a suspend/resume, or if NTP traffic is blocked — would distort every SRD number in the campaign without any user-visible symptom. The pre-flight guarantee catches this before a bad campaign starts.

IMPACT ON RERUN

Prevents a silent-corruption failure mode without cost to correct campaigns. If NTP is not sync'd, the operator sees a clear error message and can fix the sync before any data is collected. Applies to every run of every pattern.

3.1.4 run_num embedded in k6 output and event filenames

The orchestrator now writes k6 outputs as **{pattern}-run-NN-{TS}.json** and event JSONs as **{pattern}-events-NN-{TS}.json**, where NN is the two-digit run number. Previously only the timestamp appeared in the filename, and **build_master_dataset.py** assigned run numbers by directory-order enumeration.

WHY

Sorting by directory order is fragile: a re-executed failed run acquires a later timestamp and would silently shift every run number in the pattern. Embedding the run number directly in the filename makes the mapping explicit and robust to re-execution, manual file moves, and mid-campaign restarts.

IMPACT ON RERUN

build_master_dataset.py was updated in step to prefer the embedded run number when parsing filenames. Older counted-campaign filenames without the embedded number still work through a chronological-sort fallback, so the rerun data and the counted-campaign data can be re-analysed with the same pipeline.

3.1.5 Durable JSONL snapshot at end of each pattern batch

At the end of every **run-campaign.sh** pattern batch, the orchestrator now runs **kubecttl cp** to copy the watcher's persistent-volume-backed **/data/hpa-events.jsonl** file into the results directory as **hpa-events-full-post-{pattern}.jsonl**.

WHY

The counted campaign captured events only via **kubecttl logs --since-time**, which streams from the currently-running watcher pod's stdout. If the watcher pod restarts mid-campaign (it did nine times in the pilot) the earlier pod's logs vanish from **kubecttl logs**, but they remain on the persistent volume. The JSONL snapshot uses the durable persistent-volume file as the audit source of truth.

IMPACT ON RERUN

build_master_dataset.py already searches for these post-batch JSONL files by name. The snapshot means every decision the watcher ever recorded — not just those flushed to stdout — ends up in the analysis pipeline. Between-run and cross-pod-restart events that were previously ambiguous are now recoverable.

3.2 Kubernetes manifests (03-kubernetes-manifests)

Three of five manifests were changed. Namespace, service, and servicemonitor were left untouched.

3.2.1 Pod memory raised from 256 Mi to 512 Mi

Both **requests** and **limits** for the sample-app container's memory were raised from 256 Mi to 512 Mi. The CPU allocation (200m request equals limit) was left unchanged.

WHY

The Ballerina JVM's warm working set sits at approximately 115 MB idle, and JIT compilation activity under sustained load pushes the working set to 250-400 MB. The old 256 Mi limit put the pod within roughly 20 % of an out-of-memory condition at all times, and a memory spike during recalibration would silently kill the pod without producing any visible SRD or SES signal — the decision series would simply show a mysterious replica-count reset. The new limit gives approximately 2× the observed peak working set, comfortable headroom without wasting cluster memory.

IMPACT ON RERUN

Eliminates one silent-corruption failure mode. Cluster memory usage at maxReplicas of 10 rises from 2.56 GiB to 5.12 GiB, still comfortably within the 8 GiB node capacity after Prometheus, Grafana, k3s and the watcher have their share. The Ballerina JVM will also see fewer garbage-collection pauses because it has room to grow to a stable working set.

3.2.2 startupProbe added to the sample-app container

A new **startupProbe** was added to the container spec, polling **/api/health** every 6 seconds with a 5-second timeout and 10 failure attempts — giving the JVM up to 60 seconds to signal it has finished warming up before either readiness or liveness begins evaluating it.

WHY

Without a startupProbe, Kubernetes fires the liveness probe 15 seconds after container start (per **initialDelaySeconds: 15**). If the Ballerina JVM's warm-up phase runs past 15 seconds under CPU pressure, the failure counter starts against a still-warming pod. After 6 consecutive fails (≈ 60 seconds of continuously-slow health responses) Kubernetes restarts the pod — injecting a JIT-cold pod into the middle of a run and corrupting T_pod_Ready anchoring for SES. The startupProbe defers liveness evaluation until the container is genuinely ready.

IMPACT ON RERUN

Eliminates a documented silent-restart failure mode. Applies during every pod scale-up in the rerun. During the counted campaign, the risk of this triggering was small under the pilot amplitudes; with the recalibrated higher-amplitude workloads the risk was higher, and the startupProbe closes the window before it can matter.

3.2.3 Explicit HPA behavior block written out

The HorizontalPodAutoscaler manifest gained a **behavior** block that writes out Kubernetes' current defaults for both scale-up and scale-down: scale-up stabilisation window of 0 seconds with a maximum policy of 4 pods or 100 % per 15 seconds; scale-down stabilisation window of 300 seconds with a maximum policy of 100 % per 15 seconds.

WHY

The counted campaign inherited these values from Kubernetes' implicit defaults. A future Kubernetes minor version could silently change any of them, and the reproducibility claim of the thesis depends on the HPA behaviour being fixed. Writing them out explicitly guarantees that anyone reproducing the environment gets the same policy regardless of the upstream default schedule.

IMPACT ON RERUN

No behavioural change relative to the counted campaign, but significant reproducibility improvement. The 5-minute scale-down stabilisation window is what makes each run's post-k6 tail approximately 5-7 minutes long — this was implicit in the counted campaign timings and is now explicit.

3.3 HPA watcher (05-hpa-watcher)

3.3.1 Watcher memory limit raised from 128 Mi to 256 Mi

The **hpa-watcher** Deployment's memory limit was doubled. Requests remained at 64 Mi.

WHY

The Python **kubernetes** client library plus event buffers keep the watcher at 60-80 MB steady state. Under bursty scaling — for example ten pods coming Ready within thirty seconds — the working set briefly spikes. The old 128 Mi limit was close to the peak and could OOM-kill the watcher mid-run. Because the watcher's durable state lives on a persistent volume, a restart is recoverable, but each restart delays event capture and loses some stdout log context. The new limit gives approximately 3× the observed peak.

IMPACT ON RERUN

Eliminates the 'watcher restarts nine times during a campaign' behaviour observed in the pilot. Watcher OOMs were one of the reasons the JSONL snapshot at end of batch became important; even so, avoiding the OOM entirely is preferable to relying on the snapshot to recover from it.

3.4 Analysis pipeline (07-analysis)

This subsystem saw the most significant methodological changes. Six files were modified; one was renamed.

3.4.1 Direction-aware tagging fix (the main bug)

The **tag_decisions** function was rewritten so that scale-up decisions can only be tagged when their timestamp falls strictly inside a run's k6 stream window [start, end], and scale-down decisions may additionally use the extended [end, end + 15 min] tail only when no run's strict window contains the timestamp.

WHY

The original single-rule '[start, end + 15 min] with first-match' logic conflated two distinct classes of decision. During each run's load phase, a scale-up fires inside the current run's k6 stream; during each run's tail, a scale-down fires after k6 has ended but before HPA's five-minute scale-down stabilisation completes. The extended window was needed for the second case but wrongly applied to the first — because with a first-match rule, a scale-up in run N+1's k6 window that also falls in run N's extended tail matches run N first. Direction-aware tagging closes this ambiguity.

IMPACT ON RERUN

Every decision in the rerun is tagged to the correct run. On the current campaign data an empirical check confirmed 0 of 99 'extended-window-only' tags would be reassigned by the new logic — the bug was dormant under the counted campaign's specific timing but would fire the moment between-run gaps shortened. The rerun avoids the failure mode by construction.

3.4.2 CAMPAIGN_START_UTC parameterised via CLI

The hardcoded **CAMPAIGN_START_UTC** constant was replaced with a **--campaign-start** command-line argument (also readable from the **CAMPAIGN_START_UTC** environment variable). If neither is provided the pipeline falls back to the earliest run in **run_index.csv** minus six hours.

WHY

The counted campaign's value was 2026-07-10 06:30 UTC. Reruns must supply a different date. A hardcoded constant would silently include pre-rerun events in the master CSV, corrupting the run tagging.

IMPACT ON RERUN

Analysis of the rerun requires one additional flag (**--campaign-start=2026-07-13T10:00:00Z** or similar). The counted-campaign data can still be analysed by passing its original start date, giving exact reproducibility for the earlier results.

3.4.3 SLO_SUSTAINED_SAMPLES converted to SLO_SUSTAINED_SECONDS

The parameter that governs when the sustained-breach condition fires was renamed from **SLO_SUSTAINED_SAMPLES** to **SLO_SUSTAINED_SECONDS** and given a default value of 5 seconds. The **find_t_slo_risk** function's streak logic was rewritten accordingly.

WHY

The original parameter counted consecutive request samples. At high request rates (100 req/s during Burst) three samples equated to roughly 30 ms of continuous breach; at low request rates (2 req/s during Ramp's early load) three samples equated to roughly 1.5 seconds. The same parameter therefore produced different effective detection delays for different patterns, biasing cross-pattern comparisons. Time-based semantics are RPS-independent and align with the SRE Workbook's Multi-Window Multi-Burn-Rate pattern that the methodology document cites.

IMPACT ON RERUN

SRD magnitudes in the rerun will be slightly smaller than the counted campaign's because **T_SLO_risk** fires a moment later (once 5 seconds of continuous breach are confirmed, rather than 3 request samples). Cross-pattern SRD comparisons become methodologically clean. Fewer decisions will register as 'Correct but Late' because the 5-second requirement is harder to trip than the pilot's 3-sample rule at high request rates.

3.4.4 CPU-based classifier renamed to LEGACY

classify_decisions.py was renamed to **classify_decisions_v1_pilot.py** with a prominent 'LEGACY — do not use for counted campaign or rerun' banner in the docstring. **compute_srd.py** was updated to read from **master_decisions.csv** directly, skipping the CPU-based classifier from the primary pipeline. The SRD-based classifier (**classify_decisions_v2.py**) remains the primary and runs after **compute_ses.py**.

WHY

The CPU-based classifier's thresholds (30 % target, 60 % 'late') were calibrated for the pilot's 30 % HPA target. At the current 75 % target, every scale-up auto-classifies as 'Correct but Late' and every scale-down as 'Unnecessary' — the rules do not discriminate at the operating point of the rerun. Keeping it in the primary pipeline would silently produce a meaningless bucket column that could mislead future readers of the CSV. Renaming with the LEGACY banner makes the supersession explicit while preserving the file for anyone who wants to reproduce pilot-era results.

IMPACT ON RERUN

Pipeline shape becomes cleaner: **master** → **SRD** → **SES** → **classify_v2**. Rerun analysis produces one bucket column (**bucket_srd**) rather than two (**bucket** and **bucket_srd**). The counted-campaign backup CSVs still contain both columns for any comparison the thesis wishes to draw.

3.4.5 `extract_trigger_pct` handles fractional percentages

The regex used by `extract_trigger_pct` to parse the HPA's `trigger_value` string (e.g. **72% (target 75%)**) was changed to accept fractional percentages (e.g. **72.5%**).

WHY

Kubernetes' metrics-server can emit fractional percentages under certain conditions. The integer-only regex would silently drop those rows in the target filter, subtly underrepresenting real HPA decisions in the master CSV. The change is a one-line regex fix; the fractional value is coerced to a float for downstream use.

IMPACT ON RERUN

Low probability under typical HPA settings but the fix removes a hidden data-loss failure mode. Also — and this is diagnostic-only — the counted campaign's empirical check found zero fractional-percentage rows, so no historical data was actually lost, but the code is now robust for the rerun.

3.4.6 Case-insensitive `scaling_limited` check in classifier v2

In `classify_decisions_v2.py` the boolean check `str(row.get("scaling_limited")) == "True"` was replaced with a case-insensitive comparison.

WHY

CSV writers can serialise Python booleans in different cases — **True**, **true**, or **TRUE**. The original strict-case check would silently miss non-capitalised values, wrongly bucketing rows out of 'Ineffective'. Robust code should accept any case.

IMPACT ON RERUN

One-line correctness fix. Zero behavioural change on the current data (where the CSV writes **True**) but the rerun is protected against any future serialisation shift.

3.5 Sample-app (02-sample-app)

3.5.1 Explicit JVM max heap in Dockerfile

The container ENTRYPOINT now specifies **-Xmx384m**, setting the JVM's maximum heap explicitly.

WHY

Without an explicit setting, JDK 17's container-aware heuristic picks max heap as 25 % of the container memory — approximately 128 Mi on the new 512 Mi container. That is likely sufficient for the workload, but an implicit value could shift with future JDK versions or if the container memory limit is changed. Explicit 384 Mi is 75 % of the container memory, leaves 128 Mi for JVM non-heap (Metaspace, code cache, thread stacks), and provides predictable behaviour under sustained JIT compilation load.

IMPACT ON RERUN

Predictable memory profile in every rerun run. Reduces the chance of GC pauses during heap-growth phases early in the campaign. Documentation gains a clear number to cite in Chapter 3 rather than a runtime-computed heuristic.

3.6 Monitoring stack (04-monitoring)

3.6.1 Prometheus switched to persistent-volume storage

The **storageSpec** field in **prometheus-values.yaml** was changed from **{}** (implicit emptyDir) to a **volumeClaimTemplate** requesting 15 GiB on k3s's **local-path** storage class. Prometheus was recreated via **helm upgrade** to take effect.

WHY

The counted campaign relied on ephemeral emptyDir storage. Any Prometheus pod restart during the 35-hour campaign — from an OOM, node reboot, or Kubernetes eviction — would have destroyed all scraped data up to that point. HPA metrics, container CPU utilisation, and kube-state-metrics all live in Prometheus, and rerunning the analysis after the campaign requires that data to still exist. Persistent storage removes the single-point-of-failure risk.

IMPACT ON RERUN

Prometheus data now survives Prometheus pod restarts, droplet reboots, and Helm upgrades. The rerun's 30-hour runtime is now protected against exactly the kind of silent-data-loss failure the counted campaign happened to avoid by luck. Grafana screenshots for the thesis can be taken any time within the 14-day retention window.

3.6.2 Prometheus retention raised from 3 to 14 days

The **retention** field in **prometheus-values.yaml** was raised from **3d** to **14d**.

WHY

Three-day retention is tight for a research campaign. Re-querying metrics for a follow-up plot, or generating a Grafana screenshot several days after the campaign ends, would fail. Fourteen days gives comfortable margin for post-campaign analysis, thesis figure generation, and any delay in supervisor review. Disk cost at the current scrape density is approximately 5 GiB over 14 days — well within the 15 GiB PVC.

IMPACT ON RERUN

The rerun's cluster metrics remain queryable until approximately 27 July 2026. Any Grafana panel screenshots for the thesis can be generated any time before then. Combined with the persistent-volume change (3.6.1) this resolves both the durability and the retention concerns that surfaced during the audit.

3.6.3 kube-prometheus-stack chart version pinned

The **CHART_VERSION** default in **install-monitoring.sh** was changed from empty (= latest) to **83.6.0**, matching the version already deployed on the research droplet.

WHY

Different chart versions ship different Prometheus versions with different metric names and labels. A future reproduction of the environment on the same script would otherwise pick up whatever 'stable' is at the time, producing an environment materially different from the campaign's.

IMPACT ON RERUN

Reproducibility improvement only. No behavioural effect on the rerun. Anyone reproducing the setup, including the thesis defence panel or external reviewer, obtains an identical Prometheus stack.

3.7 Droplet setup (01-droplet-setup)

3.7.1 create-droplet.md size corrected to 4 vCPU / 8 GiB

The DigitalOcean droplet size documented in the setup guide was updated throughout — UI walkthrough, doctl CLI command, expected free memory and disk, and monthly snapshot cost.

WHY

The document still referenced the pilot's 2 vCPU / 4 GiB / 80 GB disk size. Anyone reproducing the environment by following the guide would end up on a droplet half the size of the actual campaign environment, and the workload amplitudes would produce entirely different HPA trajectories. This was the single most misleading documentation gap surfaced by the audit.

IMPACT ON RERUN

Reproducibility improvement. Zero direct effect on the rerun (which is already on the correct droplet). Critical for the thesis defence panel or any external reviewer who may attempt to reproduce the results.

3.7.2 k3s version pinned in install-k3s.sh

INSTALL_K3S_VERSION was set to **v1.34.6+k3s1**, matching the version installed on the droplet.

WHY

Different k3s versions ship different Kubernetes versions. Kubernetes minor versions can change the HPA's default behavior policies, the metrics-server sync period, and the container CPU accounting semantics — all of which would silently invalidate the reproducibility claim of the thesis if left un-pinned.

IMPACT ON RERUN

Reproducibility guarantee. Anyone reinstalling the environment on a fresh droplet gets the same Kubernetes version the campaign ran on. The pin can be overridden by the operator with an environment variable if a specific future test wants to use a different version.

3.7.3 Docker install supports version override

The Docker installation step in **install-k3s.sh** was wrapped so it accepts a **DOCKER_VERSION** environment variable and passes it to the get.docker.com script's **--version** flag.

WHY

Docker is used only to build the Ballerina sample-app image — the running workload uses k3s and containerd, not Docker. So the version is less critical than k3s's, but supporting an override lets a reproducer pin to a specific Docker release if strict reproduction is desired.

IMPACT ON RERUN

Reproducibility flexibility. Zero effect on the rerun. Documented in the thesis as an available override for strict-reproduction scenarios.

4. Anticipated Impact on Research Outcomes

The changes above are grouped here by the way they are expected to affect the rerun's results relative to the counted campaign. Each subsection identifies the anticipated improvement and any residual limitation.

4.1 More discriminative bucket distributions

The single strongest impact of the rerun will come from the recalibrated workload amplitudes. The counted campaign's distribution — 91 Correct & Timely, 1 Correct but Late, 90 Unnecessary, 0 Ineffective — showed that the workloads at pilot amplitudes rarely stressed the cluster enough to breach the 500 ms SLO threshold. At the recalibrated amplitudes, Step and Burst runs should reliably produce sustained CPU utilisation past 75 % on the initial 2 replicas, creating the transient during which the SRD/SES metrics are measured.

The predicted rerun distribution: approximately 50-60 % Correct & Timely for Ramp (which naturally allows HPA to scale pre-emptively), 30-40 % Correct & Timely for Step with the remainder split between Correct but Late and Unnecessary, an even split for Burst, and Noisy producing some decisions (previously zero) with the mix depending on whether HPA reacts to individual spikes.

This is speculative — the actual distribution depends on how HPA reacts to the new amplitudes on the current cluster. The important guarantee is that the Correct-but-Late bucket will no longer be near-empty, so cross-pattern comparisons become methodologically meaningful.

4.2 Trustworthy SRD numbers

The tagging fix guarantees that every scale-up decision's SRD is computed against its own run's latency trace, not the previous run's. In the counted campaign, the mistagged 71 decisions had SRD values reflecting entirely unrelated latency windows. Post-hoc correction fixed this for the counted campaign but only externally — the rerun benefits from the fix at source.

The time-based SLO-sustained parameter contributes here too. Under the sample-count-based version, `T_SLO_risk` fired between 30 ms and 1.5 s after the actual breach onset depending on request rate, so SRD numbers absorbed a variable bias. With the 5-second time-based rule, `T_SLO_risk` fires at a consistent offset relative to true breach onset in every pattern.

4.3 Valid SES windows

Because the tagging bug corrupted the run association of post-load scale-ups in Step and Burst, SES computation for those decisions used the wrong k6 file entirely — producing empty SES values in the counted campaign. In the rerun, every scale-up will be paired with its own run's latency stream and its own `T_pod_Ready` event, so the before/after windows will always contain real request data.

4.4 Reproducibility of the environment

Six of the changes above target reproducibility rather than immediate result quality: pinning the k3s version, the kube-prometheus-stack chart version, the JVM heap ceiling; writing out the HPA behavior block explicitly; correcting the documented droplet size; and embedding run numbers in output filenames. Individually none of these changes the rerun's results. Collectively they mean that anyone who wishes to reproduce the environment — the thesis defence panel, an external reviewer, or a future researcher extending the work — can arrive at a materially identical setup.

4.5 Remaining limitations

The audit did not close every limitation. Chapter 5 (Threats to Validity) of the thesis should still document:

- **Server-side latency is not independently captured.** Ballerina's Prometheus metrics reporter could not be enabled on the 2201.10.0 distribution; the 'Request latency' and 'Throughput' Grafana panels remain empty during runs. k6 client-side **http_req_duration** is the sole latency source. This is valid as a user-experience SLO measurement, but does not permit cross-verification.
- **JIT stabilisation drift.** The 3-warm-up-runs policy is conservative but not perfect; CPU-per-request may drift measurably across the campaign as JIT continues to optimise the compute loop. No independent JIT-state measurement is taken.
- **k6 remote-write not configured.** For the rerun this means Grafana cannot show live latency during runs; the operator monitors CPU and replica trajectories only. Post-campaign analysis reads the k6 JSON files directly and is unaffected.
- **Noisy pattern behaviour is uncertain in advance.** The recalibrated peaks (80-150 req/s) should trigger HPA on some spikes but not others. Whether the resulting decision count is 10, 20, or 40 will be determined only by the rerun; the paper should present the actual number as itself a flapping-resistance finding.

5. Configuration Reference (Rerun)

The following tables record the exact values used in the rerun. This section is intended for inclusion (in condensed form) as an appendix to Chapter 3 of the thesis.

5.1 Cluster and node

Parameter	Value
Droplet type	DigitalOcean Basic Premium Intel
Droplet size	s-4vcpu-8gb-intel
OS	Ubuntu 24.04 LTS x64
k3s version (pinned)	v1.34.6+k3s1
Storage class	local-path (k3s default)
kube-prometheus-stack chart	83.6.0 (Prometheus v0.90.1)
Prometheus retention	14 days
Prometheus storage	15 GiB PVC (local-path)

5.2 Sample-app deployment

Parameter	Value
Image	autoscale-sample:v5
Image pull policy	Never (locally-imported into containerd)
Container ports	9000 (http), 9797 (metrics)
CPU requests / limits	200m / 200m (equal)
Memory requests / limits	512 Mi / 512 Mi (equal)
JVM max heap	-Xmx384m
Startup probe	6 s period, 5 s timeout, 10 fails (60 s total)
Readiness probe	3 s period, 5 s timeout, 3 fails
Liveness probe	10 s period, 10 s timeout, 6 fails

5.3 HPA configuration

Parameter	Value
minReplicas	2
maxReplicas	10
CPU target (Utilization)	75 %
Memory target (Utilization)	75 %
Scale-up stabilisation window	0 s (explicit default)
Scale-up policy	Max of (4 pods, 100 %) per 15 s
Scale-down stabilisation window	300 s (explicit default)
Scale-down policy	100 % per 15 s

5.4 Analysis parameters

Parameter	Value
SLO threshold	500 ms (primary)
SLO window	30-second rolling p95
Sustained-breach requirement	5 seconds continuous
Minimum samples per SLO window	20
Warm-up runs discarded per pattern	3
Primary classifier	classify_decisions_v2.py (SRD-based)
Legacy CPU-based classifier	classify_decisions_v1_pilot.py
Threshold sensitivity range	{250, 400, 500, 750, 1000} ms

5.5 Workload amplitudes (recalibrated)

Pattern	Idle phase	Peak amplitude	Peak duration	Tail
Step	1 min	25 VUs	8 min hold	1 min
Burst	1 min	35 VUs	60 s hold	7 min
Ramp	30 s	30 VUs	8 min linear ramp	4 min
Noisy	30 s	80-150 req/s	10-30 s spikes	5 min

5.6 Watcher

Parameter	Value
Image	hpa-watcher:v3
Storage	1 GiB PVC (local-path)
CPU requests / limits	20m / 200m
Memory requests / limits	64 Mi / 256 Mi
Watch resources	HPAs, Pods, Events (read-only)
Event output	/data/hpa-events.jsonl (JSONL)

8. Timeline and Next Steps

8.1 Timeline of the audit and rerun

Date (2026)	Event
10-11 July	Counted campaign runs on 4 vCPU / 75 % target cluster
11 July	Analysis pipeline completes; 205 decisions produced; Correct-but-Late bucket essentially empty
12 July	Data-tagging bug identified and externally corrected; workload calibration mismatch identified
12 July	Supervisor correspondence: agreement to rerun at 75 % HPA target
13 July	Seven-subsystem audit conducted; 16 files modified
13 July	Calibration probe executed on current cluster; amplitudes finalised
13 July, 10:58 UTC	92-run counted rerun commences inside tmux (Step → Burst → Ramp → Noisy)
14 July, 09:22 UTC	Rerun completes (all four pattern batches); ~22.5 h end-to-end
14-15 July	Rerun analysis: build_master_dataset → compute_srd → compute_ses → classify_v2 → plots
16 July onwards (planned)	Supervisor review of rerun results; thesis Chapter 4 revision

8.2 What happens after the rerun completes

When the tmux session prints '=== FULL CAMPAIGN COMPLETE ===', the following steps run on the analysis workstation:

- **Sync results from droplet to workstation** (`scp -r ~/autoscale-decision-correctness-thesis/experiment-setup/results/ .` or equivalent).
- **Run the pipeline** in order: `python3 build_master_dataset.py --campaign-start=...`, `python3 compute_srd.py`, `python3 compute_ses.py`, `python3 classify_decisions_v2.py`.
- **Generate summary artefacts**: `overall_summary.py` for the supervisor summary CSV, and the plot scripts (bucket, SRD histogram, SES histogram, SRD×SES scatter, threshold sensitivity, bucket percentage vs. threshold, median SRD vs. threshold, case-study time series, cumulative breach count).
- **Compare against counted-campaign results** to identify changes in bucket distribution, SRD magnitudes, and SES signs. This comparison forms the empirical basis of Chapter 4.
- **Draft the supervisor summary email** with the corrected bucket table (analogous to the July 12 draft) and reference the artefacts on Google Drive.

8.3 What remains for the thesis document

Independent of the rerun, the thesis document itself requires:

- **Chapter 3** (Methodology): incorporation of the configuration reference (Section 5 above) and the methodological adjustments (SRD-based classifier, sustained-breach seconds, direction-aware tagging).
- **Chapter 4** (Results): rewrite of the results narrative using the rerun's bucket distribution. The corrected 500 ms table from the counted campaign can appear as an interim reference, but the primary results table should be the rerun's.
- **Chapter 5** (Discussion / Threats to Validity): explicit disclosure of the counted-campaign data-tagging bug and its correction, of the workload amplitude recalibration decision, and of the remaining limitations enumerated in Section 4.5 above.
- **Appendix**: this document (or a condensed version of it) would sit naturally as an appendix describing the audit and the rerun preparation.

7. Observed Rerun Results

The rerun completed on 14 July 2026 at 09:22 UTC, approximately 22.5 hours after launch (faster than the 30-hour pre-run estimate). This section presents the actual bucket distribution, compares it to the counted campaign, and summarises what the generated plots reveal per pattern. The full artefact set — CSVs, JSONL snapshots, per-run k6 outputs, batch logs, and 16 PNG plots — is on the shared drive under **rerun-2026-07-14/**.

7.1 Metric definitions: SRD and SES

The two quantitative metrics underlying the correctness classification are Scale Reaction Delay (SRD) and Scale Effectiveness Score (SES). Their definitions and the analysis windows they use are recorded here in full so the pipeline is reproducible from this document alone.

Scale Reaction Delay (SRD)

SRD is the signed time offset between the moment HPA emitted a scale-up decision and the moment the workload first sustained an SLO breach in that run:

$$\text{SRD} = T_{\text{decision}} - T_{\text{SLO_risk}} \text{ (seconds)}$$

- T_{decision} — the UTC timestamp recorded by the HPA watcher (**watcher.py**) when Kubernetes' HPA changed **desiredReplicas**. Emitted as the **detected_at** field of the **hpa_decision** JSONL event with millisecond precision.
- $T_{\text{SLO_risk}}$ — the first request-timestamp in the run at which the rolling 30-second p95 of **http_req_duration** from k6 has been continuously above the 500 ms SLO threshold for at least **SLO_SUSTAINED_SECONDS = 5 s**. Computed by **find_t_slo_risk()** in **compute_srd.py**. Windows with fewer than **MIN_SAMPLES_FOR_P95 = 20** requests are skipped to avoid spurious breach detection from single slow outliers. The function returns the timestamp of the **FIRST** sample in the sustained streak (not the confirmation moment), so SRD reflects the actual onset of the breach.

SRD interpretation:

SRD value	srd_source	Meaning
< 0	pre_emptive	HPA reacted BEFORE the breach onset (pre-emptive scaling)
0	pre_emptive	HPA reacted AT the moment of breach onset
> 0	late	HPA reacted AFTER the breach onset (late reaction)
undefined	no_slo_breach	Run had no sustained SLO breach; SRD is not applicable
undefined	scale_down	Decision was a scale-down; SRD is not defined for downscale

Scale Effectiveness Score (SES)

SES quantifies whether p95 latency dropped after HPA fired. It compares the p95 latency in a fixed window before the decision to the p95 latency in a fixed window after the newly-created pod becomes Ready:

$$\text{SES} = (\text{Latency}_{\text{before}} - \text{Latency}_{\text{after}}) / \text{Latency}_{\text{before}}$$

Both $\text{Latency}_{\text{before}}$ and $\text{Latency}_{\text{after}}$ are the p95 of k6-side **http_req_duration** in the following windows:

Window	Interval	Anchor rationale
Latency _{before}	$[T_{\text{decision}} - 60 \text{ s}, T_{\text{decision}} - 1 \text{ s}]$	60-second lookback, excluding the decision moment itself so the measurement is causally prior to the scaling action.
Latency _{after} (scale-up)	$[T_{\text{pod_Ready}}, T_{\text{pod_Ready}} + 60 \text{ s}]$	Anchored at $T_{\text{pod_Ready}}$ — NOT T_{decision} — so the ~20-40 s pod-startup window does not contaminate the measurement. This is the reviewer-approved rule in methodology §3.5.
Latency _{after} (scale-down)	$[T_{\text{decision}} + 30 \text{ s}, T_{\text{decision}} + 90 \text{ s}]$	Scale-down has no new pod. The 30-second offset gives the terminating pod time to complete its preStop grace and drain in-flight requests.
Latency _{after} (fallback, no pod_ready found)	$[T_{\text{decision}} + 60 \text{ s}, T_{\text{decision}} + 120 \text{ s}]$	Rare edge case if the corresponding pod_ready event is missing. Row is flagged with t_after_source = 'fallback' so the reader can see it happened.

$T_{\text{pod_Ready}}$ is captured by the v3 watcher: when a Pod in the sample-app namespace transitions to Ready=True, a **pod_ready** JSONL event records the transition timestamp. **pod_ready_lookup.py** matches each scale-up decision to its resulting Pod by finding the pod whose **pod_created_at** is closest to (and immediately after) T_{decision} , within a 30-second maximum gap.

SES interpretation:

- **SES > 0** — latency improved after the scaling action (HPA action was effective at reducing user-perceived latency)
- **SES = 0** — no measurable change
- **SES < 0** — latency worsened after the action. For scale-ups on a monotonically-increasing workload (Ramp), this is expected: the workload's own growth outpaces the scaling benefit. It is not automatically an HPA failure — interpretation depends on the pattern.

SES is undefined (recorded as null in the CSV) when either the before-window or the after-window contains fewer than `MIN_SAMPLES_FOR_P95 = 20` requests. This can happen near run boundaries (before-window falls outside the k6 stream) or when load ended before the after-window opens (Burst).

Where the metric values live in the CSV

Column	Meaning
srd_seconds	Signed float; positive = late, negative = pre-emptive
srd_source	Enum: 'late' 'pre_emptive' 'no_slo_breach' 'scale_down'
t_slo_risk_utc	ISO timestamp of the sustained-breach onset used for SRD
ses	Signed float; positive = latency improved
p95_before_ms	Numerator input to SES; p95 in the before-window
p95_after_ms	Numerator input to SES; p95 in the after-window
n_before_requests	Sample count in the before-window; SES null if < 20
n_after_requests	Sample count in the after-window; SES null if < 20
t_pod_ready_utc	ISO timestamp used as after-window anchor
t_after_source	'pod_ready' 'fallback' 'scale_down' — audit of the anchor choice
cold_start_delay_s	Auxiliary: $T_{\text{pod_Ready}} - T_{\text{decision}}$ in seconds

7.2 Classification criteria (SRD-based bucket rules)

Every HPA decision in the corrected pipeline is assigned to one of the four correctness buckets by **classify_decisions_v2.py** using the direction of the scaling action, the SRD signal, and the HPA **scaling_limited** condition. The rules are applied in the priority order below (first matching rule wins). This reflects the pipeline as it stands for the rerun and forward — the earlier CPU-utilisation-based rules in **classify_decisions_v1_pilot.py** are preserved only for pilot-comparability reproduction.

Direction	Condition (evaluated in order)	Bucket	reason_code
up	scaling_limited = True AND scaling_limit_reason = TooManyReplicas	Ineffective	UP_INEFFECTIVE_MAX
up	srd_source = 'no_slo_breach' (i.e. p95 latency never sustained above SLO in this run)	Unnecessary	UP_UNNECESSARY_NOBREACH
up	srd_source in {'late','pre_emptive'} AND srd_seconds is NaN	Unnecessary	UP_UNNECESSARY_NANSRD

D i r e c t i o n	Condition (evaluated in order)	Bucket	reason_code
u p	srd_source in {'late','pre_emptive'} AND srd_seconds ≤ 0 (HPA fired before or on the SLO breach)	Correct & Timely	UP_TIMELY_SRD
u p	srd_source in {'late','pre_emptive'} AND srd_seconds > 0 (HPA fired after the SLO breach)	Correct but Late	UP_LATE_SRD
u p	(fallback for any unhandled srd_source)	Unnecessarily	UP_FALLBACK
d o w n	scaling_limited = True AND scaling_limit_reason = TooFewReplicas	Ineffective	DOWN_INEFFECTIVE_MIN
d o w n	otherwise (SRD not defined for scale-downs; a premature scale-down would surface as a subsequent scale-up with SRD > 0, which is captured by the up-side rules above)	Correct & Timely	DOWN_TIMELY_SRD

How the underlying signals are computed

The rules above depend on three pipeline-derived signals per decision:

- **srd_seconds** — the Scale Reaction Delay, defined as $T_{\text{decision}} - T_{\text{SLO_risk}}$. Positive means HPA fired AFTER the SLO breach onset (late); zero or negative means HPA fired AT or BEFORE the SLO breach (pre-emptive or on-time). Computed by **compute_srd.py**.
- **srd_source** — enum recording how srd_seconds was derived: **late** (SRD > 0), **pre_emptive** (SRD < 0), **no_slo_breach** ($T_{\text{SLO_risk}}$ was never reached in this run; SRD undefined), **scale_down** (direction is 'down'; SRD not applicable). $T_{\text{SLO_risk}}$ is defined as the first request-timestamp at which the rolling 30-second p95 latency has been continuously above the 500 ms SLO for at least 5 seconds — the multi-window sustained-breach rule from the SRE Workbook.
- **scaling_limited** and **scaling_limit_reason** — read from the HPA's ScalingLimited condition. Set by **build_master_dataset.py** from the watcher-emitted hpa_conditions array. Values include TooManyReplicas (hit maxReplicas cap; classified Ineffective for up), TooFewReplicas (hit minReplicas floor; classified Ineffective for down), and DesiredWithinRange (normal operation; no Ineffective classification).

How each bucket should be interpreted

- **Correct & Timely** — HPA acted appropriately, either scaling up before the SLO would breach or scaling down once the load subsided. This is the 'good' bucket — the outcome the operator hopes to see for every decision.
- **Correct but Late** — HPA correctly identified the need to scale up, but only reacted AFTER the SLO had already breached. Non-zero end-user impact but the scaling response itself was appropriate. Ideal target for "can HPA react faster?" discussion.

- **Unnecessary** — HPA scaled up (or down; see below) even though no sustained SLO breach ever occurred in this run. For scale-ups this reflects CPU-target-driven scaling that did not correspond to a real user-experience risk — a known limitation of CPU-based HPA when latency is the SLO. This is the largest bucket for Ramp because HPA scales pre-emptively as load grows.
- **Ineffective** — HPA wanted to scale further but was blocked by a min/max replicas policy limit. For scale-ups this means the workload would benefit from a higher maxReplicas; for scale-downs it means the minReplicas floor is preventing further consolidation. Empty in the counted campaign; 30 decisions in the rerun (Burst 16, Step 12, Ramp 1, Noisy 1) reflect the higher-amplitude workloads reaching the maxReplicas = 10 ceiling.

7.3 Alternative classification: proposal-aligned v3 rules

The proposal document (Section 9, 'Decision Classification') describes a 2D SRD × SES matrix rather than the SRD-only rules implemented by `classify_decisions_v2.py`. To make the comparison explicit, an alternative classifier `classify_decisions_v3.py` was added; it implements the proposal's rules verbatim and appends its verdict to `decisions_with_ses.csv` as columns **bucket_v3**, **reason_code_v3**, **reason_text_v3**. Both classifiers coexist in the pipeline — v2 remains the primary because of the reasoning below, but v3 is available for direct comparability with the proposal.

v3 rules (from the proposal, verbatim)

Classification	SRD condition	SES condition
Correct & Timely	$SRD \leq 0$	$SES > +\tau$
Correct but Late	$SRD > 0$	$SES > +\tau$
Unnecessary	any	$ SES \leq +\tau$ (near zero)
Unnecessary	<code>srd_source = no_slo_breach</code>	any
Ineffective	any	$SES < -\tau$
Undefined (v3 only)	any	SES cannot be computed
Correct & Timely (scale-down)	n/a	n/a (SES not applicable)

where $\tau = \text{SES_NEAR_ZERO_TAU} = 0.05$ (5 % latency change treated as effectively no change). Override via env var.

Why v2 remains primary (deviation rationale)

The proposal's **Ineffective** ↔ **SES < 0** rule conflates two distinct signals: policy limits (HPA wanted to scale further but was blocked by max/min replicas) and workload shape (post-scaling latency legitimately higher than pre-scaling latency because the workload itself is still growing). Concretely:

- The Ramp pattern's mean SES in the rerun is -0.85 . This is not because HPA failed — it reflects the monotonically-increasing shape of the workload; latency 60 seconds after a scale-up is genuinely higher than 60 seconds before, because the load has grown in the interval. Under proposal rules nearly every Ramp scale-up would classify as Ineffective, hiding the pattern's textbook pre-emptive-scaling behaviour (only 3 of 72 Ramp scale-ups even breached the SLO in the after-window).
- **scaling_limited** is a clean binary signal read directly from Kubernetes' HPA ScalingLimited condition. It reports whether HPA wanted to scale further but was blocked — which is the semantic the Ineffective

bucket is trying to capture. v2 uses this signal; v3 does not.

- The proposal's **SES** ≈ 0 threshold is not quantified in the source. v3 uses ± 0.05 as a defensible default, but any specific threshold is arbitrary and would need sensitivity analysis in Chapter 5.

Both classifications are recorded in the CSV. Chapter 4 of the thesis presents the v2 (primary) bucket table as the main result and the v3 (proposal-aligned) table as the appendix, with a short paragraph noting the deviation and the reasoning.

7.4 Corrected bucket distribution

SRD-based classification (500 ms SLO, sustained = 5 s continuous) applied to 270 counted decisions. Warm-up runs 1-3 per pattern and 10 between-run decisions are excluded from this table but recorded in the full CSV for auditability.

Pattern	Correct & Timely	Correct but Late	Unnecessary	Ineffective	Total
Step	27 (57 %)	8 (17 %)	0	12 (26 %)	47
Burst	7 (16 %)	21 (47 %)	1 (2 %)	16 (35 %)	45
Ramp	52 (40 %)	3 (2 %)	75 (57 %)	1 (1 %)	131
Noisy	23 (49 %)	23 (49 %)	0	1 (2 %)	47
TOTAL	109 (40 %)	55 (20 %)	76 (28 %)	30 (11 %)	270

7.5 Comparison to the counted campaign

Bucket	Counted campaign (n = 182)	Rerun (n = 270)	Change
Correct & Timely	91 (50 %)	109 (40 %)	+18 %
Correct but Late	1 (0.5 %)	55 (20 %)	+55× (biggest change)
Unnecessary	90 (49 %)	76 (28 %)	−16 %
Ineffective	0 (0 %)	30 (11 %)	new signal — ceiling saturation

Two changes dominate. The Correct-but-Late bucket rose from 1 to 55 decisions — the bucket taxonomy is now genuinely discriminative for the paper. The Ineffective bucket, empty in the counted campaign, now contains 30 decisions concentrated in Burst (16) and Step (12) where sustained load pushes HPA into the `maxReplicas = 10` ceiling. Noisy went from zero decisions to 47 — the recalibrated amplitude successfully triggers HPA on some spikes, exactly as the pattern was designed to test.

7.6 Per-pattern observations from the plots

Ramp — clean pre-emptive scaling

The ``latency_around_hpa_decisions.png`` panel for Ramp shows only 3 of 72 scale-ups exceeded 500 ms in the post-decision window. Median p95 latency across all 72 Ramp scale-ups stays between 200 and 400 ms in the entire $[T_{\text{decision}} - 60 \text{ s}, T_{\text{decision}} + 180 \text{ s}]$ window — well below the SLO. In ``case_study_timeseries.png``, ramp-04 shows HPA scaling $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ replicas as the gradual load rises, and the SLO is never crossed. This is the textbook pre-emptive-scaling case and validates that HPA at industry-standard operating points handles slowly-rising load without incident.

Burst — reactive with quick recovery

19 of 20 Burst scale-ups exceeded 500 ms in the post-decision window. However, the median latency curve returns below the SLO within approximately 50 seconds — the load itself ends before HPA fully scales. The exemplar burst-04 shows p95 briefly spiking to ~700 ms then dropping to zero as the 60-s hold ends. Burst's Ineffective count (16 of 45, 35 %) reflects the frequency with which HPA's scale-up limit was reached before the transient ended — a real observation about how HPA copes with sub-stabilisation-window transients.

Step — partial reaction, plateau near SLO

17 of 20 Step scale-ups exceeded 500 ms in the post-decision window; unlike Burst, Step's median latency stays at the SLO boundary rather than dropping below it. The exemplar step-04 shows HPA scaling only $2 \rightarrow 3$ during the 8-minute load hold, so latency plateaus at 500–600 ms for the remainder of the run. Step's Ineffective count (12 of 47, 26 %) captures the runs where HPA eventually reaches `maxReplicas = 10` and can scale no further.

Noisy — workload-amplitude finding

All 20 Noisy scale-ups exceeded 500 ms after the decision, and median latency reached 1500–2000 ms sustained across the spike periods (visible in both plots). HPA reached only 3 replicas in the noisy-04 exemplar — the metrics-server averaging window (~15 s) is too slow to react to Noisy's brief 10–20 s peaks, so HPA scales conservatively while instantaneous CPU exceeds 100 % of the initial 2 replicas' requests. This is a legitimate finding — HPA at industry-standard operating points cannot cope with brief high-amplitude spikes — but the amplitude is also an over-tune of the recalibration decision. A follow-up run at 40–80 req/s peaks (roughly half the current 80–150 range) would produce a milder flapping-resistance test if isolating that property is desired. Both are worth documenting in Chapter 5 as an amplitude-vs-flapping-resistance trade-off.

7.7 Quality flag: Noisy dropped iterations

Every Noisy run in the rerun triggered the orchestrator's dropped-iterations quality flag with values between approximately 1500 and 4100 per run. k6's ramping-arrival-rate executor could not always sustain the requested 80–150 req/s peaks against the 60-VU preallocated pool once server response time drifted up under load. Drops are documented per-run in the batch log `logs/noisy-batch-*.log`. This does not affect the recorded HPA decisions (the drops are on the load-generator side, not on requests that reached the sample-app) but is a documentable workload-generator sizing consideration for Chapter 5.

7.8 Which of Section 4's predictions were confirmed

Comparing the anticipated impact (Section 4) to the observed results:

- **4.1 More discriminative bucket distributions:** confirmed. Correct-but-Late $1 \rightarrow 55$; Ineffective $0 \rightarrow 30$.

- **4.2 Trustworthy SRD numbers:** confirmed. Direction-aware tagging produced 10 legitimate between-run decisions correctly labelled (versus 1 stray in the counted campaign) — the fix is earning its keep by surfacing what was previously mis-attributed.
- **4.3 Valid SES windows:** confirmed. Every scale-up decision was paired with its own run's latency stream and its own T_pod_Ready event; no empty SES values from the tagging bug of the counted campaign.
- **4.4 Reproducibility improvements:** confirmed. The environment is fully pinned (k3s v1.34.6+k3s1, chart 83.6.0, explicit HPA behavior block, explicit JVM heap, documented droplet size).
- **4.5 Remaining limitations:** Ballerina Prometheus reporter still not enabled (k6 remains sole latency source); JIT stabilisation drift still present but managed by 3-warm-up policy. NEW limitation from the rerun: Noisy amplitude tuned too aggressively — both a finding and a documented trade-off (see Section 7.3 Noisy).

7.9 Threats to validity

The proposal document identifies three categories of threat to validity. This subsection records how each is addressed by the audited pipeline and where residual risk remains. This text lifts almost verbatim into Chapter 5 (Threats to Validity) of the thesis.

Internal validity

Threats: confounding factors that could produce the observed HPA behaviour without reflecting the true correctness-vs-workload relationship the study aims to characterise.

- **Controlled workloads (addressed).** All four workload patterns are driven by fixed k6 scripts with recalibrated amplitudes documented per pattern (Sections 3.1, 5.5). No human traffic is mixed in; no A/B experimental variation is introduced during a run. Each pattern's shape is deterministic.
- **Fixed HPA policy (addressed).** Every rerun run uses the same HorizontalPodAutoscaler spec (Section 5.3). The behavior block is written out explicitly (Section 3.2.3) so no future Kubernetes upgrade can silently shift the observed policy.
- **Between-run cross-contamination (addressed).** The orchestrator's `wait_for_idle` logic requires HPA to return to **minReplicas = 2** and **CPU < 5 %** for at least 60 continuous seconds before starting the next run. This guarantees each run begins from the same baseline.
- **Clock drift (addressed).** A pre-flight NTP-sync check (Section 3.1.3) refuses to start a run if the droplet clock is not synchronised. This closes a documented silent-corruption failure mode where SRD numbers would drift with clock offset.
- **Residual risk: JIT stabilisation drift.** The 3-warm-up-runs-per-pattern policy is conservative but not perfect. CPU-per-request may drift measurably across a 20-run counted batch as JIT continues to optimise the compute loop. No independent JIT-state measurement is taken. Disclosed as Section 4.5, third bullet.

External validity

Threats: aspects of the experimental environment that limit generalisation of findings to other setups.

- **Focus on behaviour, not absolute performance (addressed).** The correctness classification is a per-decision categorisation into four buckets. The bucket assignments generalise across clusters of different absolute scale — they characterise HPA behaviour patterns rather than latency numbers. A reproducer on a differently-sized cluster should observe similar behavioural signatures (e.g. Burst mostly Late, Ramp mostly Timely) even if the absolute SRD magnitudes differ.
- **Single-node k3s cluster (limitation).** The rerun runs on a 4 vCPU / 8 GiB DigitalOcean droplet with k3s. Multi-node clusters may see different pod-scheduling latency (which is distinct from HPA reaction

latency) and different CPU-metric-aggregation behaviour. Sections 5.1 and 3.7.1 document the environment.

- **Industry-representative operating points (addressed).** The 75 % CPU HPA target is defensible per Google's GKE engineers (Section 3.7.1 and email correspondence with Suvin). Findings should therefore reflect what a production Kubernetes deployment at industry-standard targets actually experiences.

- **Residual risk: single-workload-family.** The four patterns exercise transient, sustained, gradual, and stochastic load respectively, but they all invoke the same CPU-intensive endpoint. Real-world workloads mix CPU, memory, and I/O in ways this study does not cover.

Construct validity

Threats: whether the operational definitions of SRD and SES faithfully capture the theoretical concepts they are meant to measure — namely HPA timeliness and HPA effectiveness.

- **SRD tied to actual SLO breach (addressed).** T_SLO_risk is derived from k6-observed p95 latency (30-s rolling window, 5-s sustained-breach rule from the SRE Workbook Multi-Window Multi-Burn-Rate pattern). SRD directly measures the temporal gap between the SLO-breach onset and the HPA action — which is exactly the 'timeliness' construct.

- **SES tied to real before/after latency (addressed).** Both Latency_before and Latency_after are p95 of client-side http_req_duration in windows anchored to observable events (T_decision, T_pod_Ready). The T_pod_Ready anchor prevents cold-start delay from contaminating Latency_after — the reviewer-approved rule in methodology §3.5 addresses a documented construct threat.

- **Metric derivation from real system outcomes (addressed).** Every metric value is traceable to raw observation data (k6 http_req_duration JSON stream, watcher-emitted hpa_decision and pod_ready JSONL events). Nothing is synthesised. The durable JSONL snapshot per pattern batch provides an audit trail (Section 3.1.5).

- **Residual risk: server-side latency not independently captured.** Ballerina's Prometheus reporter is not enabled on the current distribution; k6 client-side latency is the sole source. This is a documented construct-validity limitation (Section 4.5, first bullet) that a Chapter 5 paragraph should discuss explicitly.

Analytical methods coverage

The proposal identifies four analytical methods. Coverage in the current pipeline is:

Proposed method	Coverage in the audited pipeline
Descriptive statistics (mean, variance of SRD and SES)	Mean, median, min, max in ses_summary.csv and SRD console output. Variance is not explicitly computed but is a one-line addition if the thesis requires it.
Time-series visualization	case_study_timeseries.png shows per-pattern exemplar runs (p95 latency + replica count + SLO threshold + T_SLO_risk / T_decision markers).
Temporal correlation analysis	plot_temporal_correlation.py was added post-audit. It correlates SRD magnitude with time-within-run (Pearson r + OLS regression per pattern) and plots per-run cumulative-decision-count trajectories. Output: temporal_correlation.png .

Proposed method	Coverage in the audited pipeline
Comparative analysis across workload types	Every summary CSV, every plot, and Sections 7.4 and 7.6 of this document present per-pattern comparisons. This is the most thoroughly covered method.

Appendix A: File Inventory of Changes

The following files were modified between 12 and 13 July 2026 in preparation for the rerun. All changes are committed to the project repository.

Subsystem	File	Type of change
01-droplet-set up	create-droplet.md	Doc: 2 vCPU/4 GB → 4 vCPU/8 GB
01-droplet-set up	install-k3s.sh	k3s v1.34.6+k3s1 pinned; Docker override
02-sample-app	Dockerfile	-Xmx384m explicit JVM heap
03-kubernetes-manifests	01-deployment.yaml	Memory 256 → 512 Mi; startupProbe
03-kubernetes-manifests	03-hpa.yaml	Explicit behavior block; comment refresh
04-monitoring	prometheus-values.yaml	PVC 15 GiB; retention 3d → 14d
04-monitoring	install-monitoring.sh	Chart version 83.6.0 pinned
05-hpa-watcher	watcher-deployment.yaml	Memory limit 128 → 256 Mi
05-hpa-watcher	README.md	v1 → v3; target 50% → 75%
06-load-tests	run-campaign.sh	NTP check; run_num in filename; JSONL snapshot
06-load-tests	calib-probe.js	Refresh; stage range 5-40 VUs
06-load-tests	step-load.js	Peak 15 → 25 VUs
06-load-tests	burst-load.js	Peak 25 → 35 VUs; supersede 3 July note
06-load-tests	ramp-load.js	Peak 20 → 30 VUs
06-load-tests	noisy-load.js	Peaks 22-35 → 80-150 req/s
06-load-tests	README.md	Rewrite of stale sections
07-analysis	build_master_dataset.py	Direction-aware tagging; CLI arg; regex fix

Subsystem	File	Type of change
07-analysis	compute_srd.py	Reads master directly; SLO_SUSTAINED_SECONDS = 5
07-analysis	classify_decisions.py	Renamed to classify_decisions_v1_pilot.py; LEGACY banner
07-analysis	classify_decisions_v2.py	Case-insensitive scaling_limited

Appendix B: Glossary

Selected terms used throughout this document.

Term	Definition
HPA	HorizontalPodAutoscaler — Kubernetes controller that adjusts the number of replicas of a Deployment based on observed metrics (typically CPU or memory).
SRD	Scale Reaction Delay — $T_{\text{decision}} - T_{\text{SLO_risk}}$. Positive: HPA reacted after the SLO breach (late). Negative: HPA reacted before the SLO breach (pre-emptive).
SES	Scale Effectiveness Score — $(\text{Latency_before} - \text{Latency_after}) / \text{Latency_before}$. Positive: latency improved after the scaling action. Negative: latency worsened (typical of monotonically-increasing workloads like Ramp).
$T_{\text{SLO_risk}}$	First timestamp in a run at which the rolling 30-second p95 latency has been above the SLO threshold continuously for at least <code>SLO_SUSTAINED_SECONDS</code> (default 5 s).
$T_{\text{pod_Ready}}$	Timestamp at which a newly-created Kubernetes Pod's Ready condition transitions to True. Used as the anchor of the SES after-window so cold-start delay does not contaminate <code>Latency_after</code> .
MWMBR	Multi-Window Multi-Burn-Rate. An SLO-alerting pattern from Google's SRE Workbook that requires a sustained-time-based breach detection rule, cited by the methodology and reflected in the 5-second <code>SLO_SUSTAINED_SECONDS</code> parameter.
Bucket taxonomy	Four-class classification of HPA decisions: Correct & Timely, Correct but Late, Unnecessary, Ineffective. Assigned by <code>classify_decisions_v2.py</code> from SRD, <code>srd_source</code> , and <code>scaling_limited</code> signals.
Warm-up runs	First three runs of each pattern batch, discarded from analysis per methodology. In practice they allow HPA state, JVM JIT compilation, and Prometheus scrape queues to reach steady state before counted-run measurements begin.
Counted campaign	The 10-11 July 2026 92-run campaign whose 206 decisions produced the pilot-adjacent results table (Section 2.1). Superseded by the ongoing rerun as the thesis's primary data.